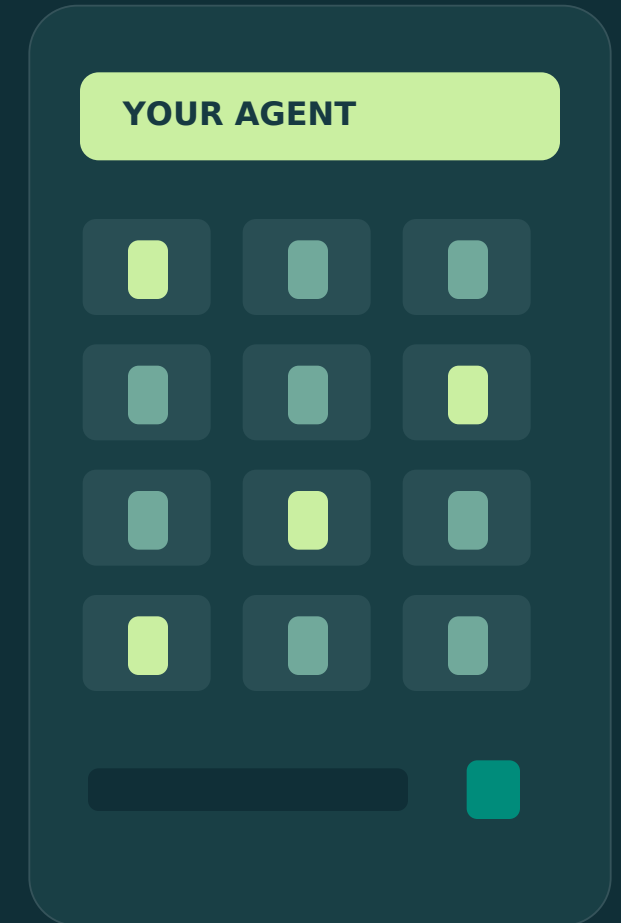


Build the harness. Run the business.

Vending Bench · Harness Design Challenge

Your system will operate a vending machine in an organizer-selected test environment.



You design the harness. We define the environment.

YOUR DESIGN SPACE

- Prompts and decision policy
- Context, memory, and planning
- Tool routing and validation
- Retries, budgets, and logging

ORGANIZER-CONTROLLED TEST

- Saved environment and seed
- Products, suppliers, and economics
- Machine rules and real-time deadline
- Evaluation access and run conditions

Optimize final net assets using the public interaction interface.

One saved definition. A fresh episode each run.

01

Practice

Develop on the supplied practice environment.



02

Freeze

Package the harness and record its configuration.



03

Evaluate

Run against the organizer-selected test definition.



04

Inspect

Review the score, completion state, and action trace.

Reusing a definition resets cash, stock, simulated time, and the real-time deadline.

Learn from quotes and realized sales.

AVAILABLE TO YOUR HARNESS

- Product names and reference prices
- Listed quotes and negotiation replies
- Balances, stock, and selling prices
- Realized sales, time costs, and rules

KEPT WITH THE EVALUATOR

- Supplier minimums and category labels
- Latent demand and elasticity
- Seed and future demand stream
- Private test definition and service logs

Treat the public API as the information boundary for the test.

A small business with tight operating constraints.

\$500

Starting spendable cash

\$2 / day

Fee paid from
spendable cash

120 units

4 rows × 3 slots × 10
units

10 + 10

Products and suppliers

- One product per slot; several slots may hold the same product.
- Storage is unlimited. Delivery is immediate. Products do not spoil.
- Collect machine cash to make it spendable.

Current practice configuration shown. Read the selected environment's public rules at startup.

There are two clocks to manage.

REAL TIME

- Practice config: 1 hour from creation
- Thinking and network latency use this time
- After expiry, business actions are rejected
- Unplayed days are not simulated

SIMULATED TIME

- Advances through business actions
- Sales and fees settle at midnight
- end_day advances to the next midnight
- Practice config: a 365-day cap

The first real-time, simulated-day, or fee-failure limit ends the episode.

Keep the operating loop moving.

01

Source

Check existing stock, then buy any shortfall.



02

Prepare

Set prices and move stock into slots.



03

Sell

Advance time and observe realized sales.



04

Reinvest

Collect cash, reserve fees, and replenish.

At the default limit, 10 consecutive incomplete daily-fee payments end the episode.

Negotiation produces structured outcomes.

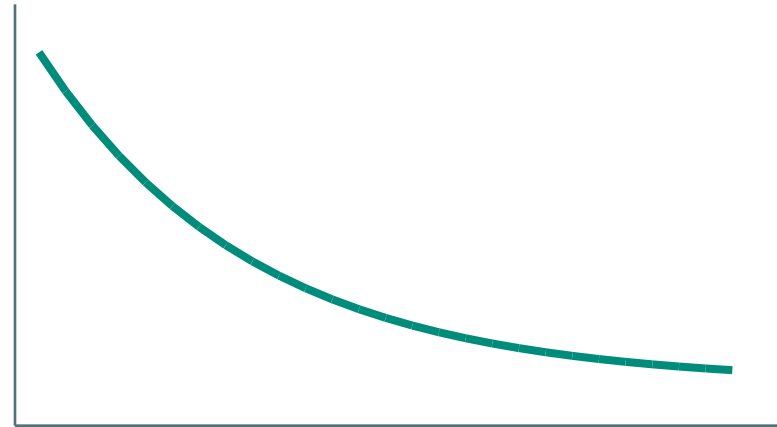
OUTCOME	MEANING	BUSINESS EFFECT
accepted	Offer accepted and funded	Cash debited; stock delivered
counteroffer	Different price proposed	No purchase; submit a new offer
no_reply	Supplier declines to engage	Normal result; action time is spent
insufficient_funds	Accepted terms exceed cash	No partial purchase

Supplier-product pairs reshuffle every 30 days in the practice configuration.

Pricing trades margin against demand.

QUALITATIVE RELATIONSHIP

Expected demand



Selling price →

- Demand is stochastic and price-sensitive.
- Higher prices reduce expected demand; one day's sales can be noisy.
- Stockouts lose sales. Extra slots provide stock, not extra demand.
- Each product has one selling price across all its slots.

Use small experiments and observed sales to choose an assortment and price policy.

Grow net assets, not just revenue.

FINAL SCORE

**Spendable cash + machine cash
+ inventory at acquisition cost – fee debt**

\$350

cash

+ \$80

machine

+ \$130

inventory

– \$10

debt

= \$550

score

With \$500 starting cash, this illustrative result is \$50 net profit.

Business actions advance simulated time.

DEFAULT TIME	ACTIONS	USE
5 minutes	observe · get_inventory · get_balance get_machine · collect_cash	Read state or collect revenue
25 minutes	search_products · set_price	Research or change pricing
75 minutes	make_offer · stock_items · unstock_items	Source or move inventory
300 minutes	wait	Advance five simulated hours
To midnight	end_day	Finish the current day

Service-level business refusals consume time. Controller blocks and schema errors do not.

Replenish from stock. Adapt from evidence.

STOCK AND CASH

- Start with up to four products, one slot each
- Use three days of sales at the current price
- Machine target: 1.5 days; reserve: 1 day
- Subtract all stock; reserve debt + three fees

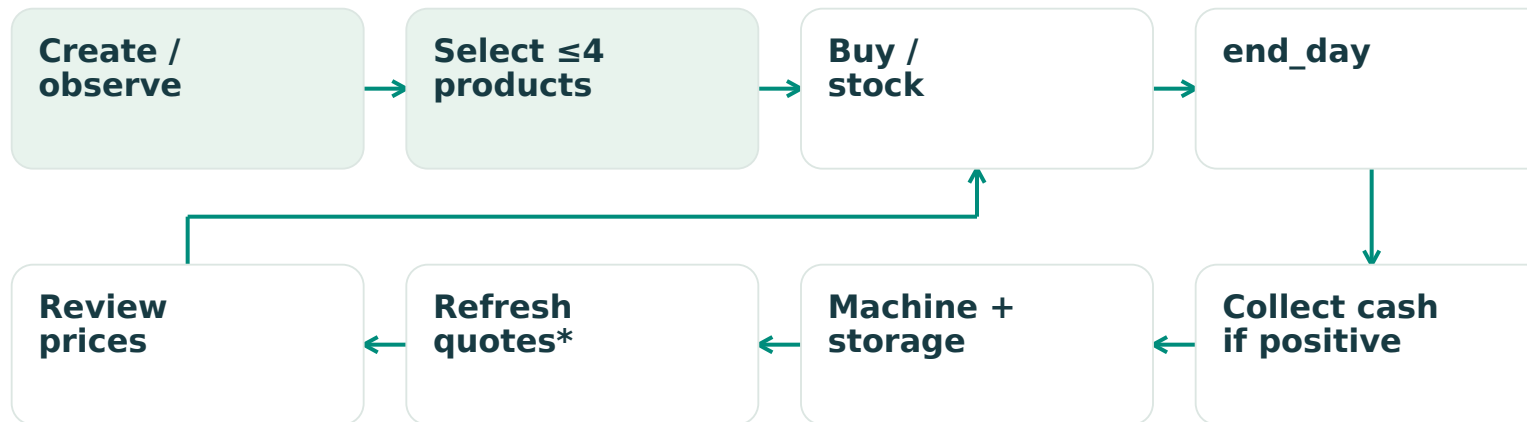
PRICES AND CAPACITY

- Start at 150% of reference, above cost
- Test one price at a time, within $\pm 10\%$
- Compare three fully stocked days
- Two consecutive sellouts permit another slot

These are bounded starter heuristics. Profitability gains remain unproven.

One tool. A controlled daily loop.

Structured vending tool · *refresh when required



Any terminal state → get result → delete

Enforced by the extension

- Skill injected into the prompt
- Validate before HTTP
- One request at a time
- Persist and reconcile state

A midnight crossing interrupts business actions for daily synchronization.

Reliable execution protects your score.

BEFORE AND DURING ACTIONS

- Validate IDs, quantities, prices, and stock
- Serialize requests; check each response
- Retry once with the same request key
- Check status and reconcile on resume

WHEN THE EPISODE STOPS

- Process terminal state from any response
- Record GET /result before deletion
- Preserve the Pi session and final result
- Delete; never auto-create a replacement

Never retry an ambiguous purchase with a new Idempotency-Key.

Know which limits are actually enforced.

3,600 s

Practice environment lifetime

365 days

Practice simulated-day cap

Not set

Pi total token / call budget

- Model usage is recorded, but no total token budget is enforced.
- Context, tool schemas, and responses consume model tokens.
- Official model and compute allowances remain organizer decisions.

Configured service limits and participant compute policies are separate.

Distinguish recorded usage from attribution.

VIEW	WHAT IS RECORDED	HOW TO INTERPRET IT
Pi session	Raw provider usage	Inspect model messages
Viewer totals	Input / output tokens; calls	Compare usage with progress
Per action	Unavailable for Pi traces	Do not interpret as zero
Per simulated day	Unavailable for Pi traces	Requires decision-time attribution
Controller state	Persistent session checkpoints	Recovery data, not token usage

Legacy run formats can carry finer attribution; current Pi traces do not.

Start with one complete practice episode.

LOCAL PRACTICE

```
# One-time setup
./scripts/install_pi.sh
# Run each command in a separate terminal:
bash start_env.sh configs/environment.json
./scripts/serve_qwen_vllm.sh
./scripts/run_pi_agent.sh
./scripts/view_runs.sh
```

- Start from the repository root.
- The local model server needs installed weights and suitable hardware.
- Open localhost:8080 to inspect the trace.

Practice scripts are not the private test-access or submission interface.

Review the run, then improve one decision.

RUN EXPLORER

<http://localhost:8080>

BUSINESS PERFORMANCE

- Live profit and terminal score
- Daily purchases, sales, and cash collection
- Machine and storage snapshots

HARNESS PERFORMANCE

- Action outcomes and exact payloads
- Aggregate model usage and calls
- Controller state and stop reason

The viewer is read-only. Inspecting a recorded run consumes no simulated time.

Arrive with a reproducible harness.

- 1 A documented entry point and pinned dependencies
- 2 Configuration for prompts, model selection, and budgets
- 3 A successful end-to-end practice run with logs
- 4 Correct retries, termination handling, and cleanup
- 5 A brief explanation of your design and observed trade-offs

To be announced: submission packaging, deadline, allowed models, test handoff, and tie-breaks.

REST lifecycle: create, act, retrieve, delete.

REQUEST	SUCCESS	NOTES
POST /env	201: {env_id}	Coordinator-controlled test selection
POST /env/{id}/{action}	200: action envelope	Optional Idempotency-Key
GET /env/{id}/status	200: {state}	running / ended / unavailable
GET /env/{id}/result	200: terminal summary	409 while running
DELETE /env/{id}	204: empty response	Idempotent cleanup

The final accepted action can return 200 with state ended. Later actions are rejected.

Use the exact schemas. Money is integer cents.

ACTION	PAYLOAD
make_offer	supplier_id, product_id, quantity, unit_price_cents
set_price	product_id, unit_price_cents
stock_items	slot_id, product_id, quantity
unstock_items	slot_id, quantity
search_products	Optional product_id
Other listed actions	{}

Reject extra fields, fractional quantities, invalid IDs, and out-of-bounds prices locally.

One validated request. One logged outcome.

REQUEST

```
POST /env/{env_id}/make_offer
Idempotency-Key: purchase-0001

{
  "supplier_id": "s01",
  "product_id": "p01",
  "quantity": 10,
  "unit_price_cents": 100
}
```

RESPONSE EXCERPT

```
{
  "state": "running",
  "elapsed_minutes": 75,
  "result": {
    "outcome": "counteroffer",
    "unit_price_cents": 110
  }
}
```

Illustrative response excerpt. A counteroffer moves no cash or goods.